



MAKERERE UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATION SCIENCES

BSE4104: EMERGING TRENDS IN SOFTWARE ENGINEERING

AI-NATIVE & AGENTIC ENGINEERING CAPSTONE: WEEK 1 SUBMISSION

PROJECT: UNIVERSITY STUDENT-SUPPORT CASE AGENT

STUDENT NAMES	STUDENT NO.	REGISTRATION NO.
KABANDA ARTHUR	2300708510	23/U/08510/PS
TUMUKUNDE KATO ANDREW	2300718082	23/U/18082/PS
GARANGA JOHN	2300722975	23/X/22975/PS
ARIKO SOSSY JOEL	2300706399	23/U/06399/EVE

Table of Contents

1. Project Overview and Rationale	3
2. Project Charter.....	3
2.1 Problem	3
2.2 Target User	3
2.3 Current Pain Point	4
2.4 Why AI Adds Value	4
2.5 Scope: Single Primary End-to-End Workflow	4
2.6 Assumptions	5
2.7 Constraints.....	5
2.8 Minimum Proposal Statement	5
2.9 Success Criteria	5
2.10 Key Risks and Mitigations	6
3. User Stories and Acceptance Criteria.....	6
4. AI Boundary Matrix.....	7
5. Initial Architecture / Context Diagram.....	7
6. Way Forward: Week 1 Roles and Task Breakdown.....	8
6.1 Role Assignment	8
6.2 Immediate Task List (due 4 September 2026).....	8

1. Project Overview and Rationale

We selected the University Student-Support Case Agent from the approved use-case list in the assignment brief. The choice was driven by three practical factors for an eight-week, four-person team:

- Data is genuinely obtainable in Week 1 course outlines, the student handbook, and academic-regulations documents are public university material; anything sensitive is replaced with synthetic student/case records.
- The workflow is naturally multi-step and tool-using (retrieve → check case status → create/route a ticket → remember the case), which satisfies the brief's rule that the system must do more than "wrap a chatbot."
- The safety boundary is unambiguous: the agent may explain, retrieve and draft, but admissions, grading, disciplinary and fee decisions are excluded entirely and remain human-only — easy to defend at every weekly review and in the final demo.

The system helps a Makerere student get grounded answers about courses and university procedures, check the status of an existing support case, and raise a new case when self-service does not resolve their issue — without waiting on a manual front desk for routine queries.

2. Project Charter

2.1 Problem

Makerere students routinely need answers to procedural questions — registration deadlines, retake and resit rules, timetable clashes, course-add/drop windows, examination policy, how to formally raise an issue with the department. This information exists, but it is spread across the student handbook, individual course guides, and academic-regulations documents, often as long PDFs with no search function tuned to plain-language questions. A student who wants to know "can I still register two weeks late" has to know which document to open, find the right section, and interpret formal regulatory language correctly — a process that is slow even when the answer is unambiguous, and error-prone when it isn't.

A related but distinct problem sits on the case-handling side. Once a student has raised an issue with the department office (a registration problem, a timetable conflict, a grading query routed for review), there is currently no way to check progress except by emailing or physically visiting the office. This creates two costs at once: the student loses time and certainty waiting for a reply, and staff time is consumed answering "any update?" queries rather than progressing the underlying case. In effect, the same manual channel is used both for genuinely new issues and for simple status checks, which slows both down.

2.2 Target User

Primary user: undergraduate student (initially scoped to CoCIS): a student who needs two related things: fast, trustworthy answers to handbook/course questions without hunting through PDFs, and a lightweight way to either check an existing case or raise a new one without waiting on office hours or email response times. This user is comfortable with a chat interface, values clear yes/no or cited answers over long explanations, and needs to be able to tell when the system genuinely doesn't know something rather than receiving a confidently wrong answer.

Secondary user: support-office staff: staff who currently spend time on first-line triage — reading an incoming query, classifying what kind of issue it is, and either answering directly or forwarding it. This user benefits indirectly: tickets that arrive already categorised and summarised are faster to act on, and routine status queries no longer need staff time at all.

Tertiary stakeholder: course convener / module reviewer: not a system user, but the person the team is accountable to for demonstrating that the system behaves safely and predictably at each weekly review.

2.3 Current Pain Point

The underlying issue is an access gap, not an information gap: the answers usually already exist, but not in a form a student can query in the moment they need it. Three concrete symptoms follow from this. First, students default to asking a human (staff, course mates, or a departmental WhatsApp group) rather than the source document, because it's faster even when less reliable. Second, there is no self-service status channel, so every case — however routine — generates at least one follow-up contact. Third, staff time is spent disproportionately on repetitive first-contact triage rather than on resolving the harder cases that actually need a human's judgement.

2.4 Why AI Adds Value

A rule-based FAQ bot or a simple keyword search over the handbook would only handle questions phrased exactly the way the documents are written, and would break on paraphrases, follow-up questions, or anything not explicitly indexed. A foundation model grounded in the handbook/course corpus can instead interpret a question asked in a student's own words, retrieve the genuinely relevant section even when the phrasing doesn't match, and produce a plain-language answer with a citation back to the source combining the flexibility of natural language with the traceability of a real document reference.

The same language capability is useful on the ticket side: a student's free-text description of a problem ("my exam clashes with another paper and nobody told me which room to go to") is naturally messy, and turning that into a clean category and summary is a task language models are well suited to, but that a rigid form would handle poorly by forcing the student into predefined boxes that may not fit.

Importantly, AI is used specifically where language understanding adds value for example interpreting questions, retrieving evidence, and drafting summaries and nowhere else. Every factual data point that matters (a case's actual status, a ticket's ID, which queue it's routed to) is still produced and controlled by deterministic software. This split is what keeps the system trustworthy: the model's flexibility is used for language, not for facts.

2.5 Scope: Single Primary End-to-End Workflow

A student opens the chat interface and asks a question. If it is a knowledge question, the agent retrieves grounded evidence from the handbook/course corpus and answers with a source citation, or states plainly that it could not find an answer in the available documents rather than guessing. If the question concerns an existing case, the agent calls a tool to look up the case by ID and reports the status exactly as returned — never a paraphrase that could drift from the real value. If the student's issue is unresolved or new, the agent drafts a category and summary from their description and creates a ticket through a tool, returning a ticket ID; any ticket flagged by the routing rules as high-impact is held for staff sign-off before being finalised. Throughout the conversation, the agent remembers the active case ID for that session, so the student is not asked to repeat it once it has been given.

Out of scope for this project: authentication against a real student-records system, integration with a live university database, admissions/grading/disciplinary/fee decisions of any kind, multi-department routing beyond the pilot department, and any workflow beyond the single primary path described above. These exclusions are deliberate, they keep the eight-week build achievable and keep every included capability demonstrably safe.

2.6 Assumptions

The team can obtain or reconstruct a public course/handbook corpus of roughly 10–30 documents by Week 3; where a document isn't publicly available in time, a synthetic equivalent will be created and clearly labelled as such.

- All student and case records used for development and testing are synthetic or team-created; no real student data is used at any point.
- A single foundation model, selected in Week 2 against cost, latency and reliability criteria, is sufficient for this scope; no fine-tuning is required.
- The pilot is scoped to CoCIS to keep the corpus and case categories manageable within the timeline; the design should not preclude extending to other colleges later

2.7 Constraints

- No admissions, grading, disciplinary or fee decisions may be made or influenced by the agent — these remain entirely human, deterministic processes outside the system's action space, enforced at the orchestrator level rather than left to the model's judgement.
- The case-status and ticket-creation tools operate against a mock/sandbox data store for the duration of the module; no connection to a real university system is in scope.
- The team has eight weeks with fixed weekly deliverables, so scope is deliberately limited to one workflow, one corpus, two initial tools, and one bounded agent loop rather than a broader multi-department system.
- Team capacity is four members across five conventional roles, so two roles are merged (Section 6.1); this is factored into the task breakdown rather than treated as a gap.

2.8 Minimum Proposal Statement

Our system helps a Makerere student complete grounded handbook/course questions, case-status checks and support-ticket creation. AI is used for reasoning, generation and retrieval — interpreting the student's question, retrieving relevant handbook/course evidence, and drafting ticket summaries. Deterministic software remains responsible for authentication, case-status data, ticket routing rules and storage. The agent may use the retrieval tool, `get_case_status` and `create_support_ticket` tools, but may not make or influence admissions, grading, disciplinary or fee decisions. We will build and evaluate the system using public Makerere course/handbook documents and synthetic student and case records.

2.9 Success Criteria

The Week-1-to-8 build will be considered successful if: a student can get a correctly sourced answer to an in-corpus handbook question; the agent visibly declines to answer questions outside the corpus rather than fabricating a response; a case-status check always reflects the tool's actual returned value; a new ticket can be created end-to-end with a sensible category and summary; and the system correctly refuses or redirects any question touching the excluded categories in Section 2.5, every time, across the team's test scenario set.

2.10 Key Risks and Mitigations

The corpus may be smaller or lower-quality than planned if public documents are hard to source in time — mitigated by starting corpus scouting in Week 1 (Section 6.4) and allowing synthetic documents to fill gaps. The model may hallucinate an answer despite retrieval — mitigated by explicit prompting to answer only from retrieved evidence and by a dedicated "unanswerable questions" test set from Week 3 onward. Scope creep toward a broader ticketing system is a risk given four workflows are conceptually adjacent — mitigated by the Out-of-Scope list above being treated as a hard boundary, not a suggestion.

3. User Stories and Acceptance Criteria

Twelve testable user stories covering the four Week-1-relevant roles: student (primary user), support-office staff, and the QA/security lead perspective the brief asks teams to hold internally.

ID	As a...	I want to...	So that...	Acceptance Criteria
US-1	Student	ask about a registration or academic-regulation procedure and get a grounded answer	I trust the answer and don't have to search PDFs myself	Answer cites the specific handbook/course document and section; if not found, the agent says so explicitly rather than guessing.
US-2	Student	check the status of my existing support case using my case ID	I don't need to email or visit the office for a routine update	Given a valid case ID, the agent returns the deterministic status from the tool, not a generated guess.
US-3	Student	be told clearly when the agent doesn't know something	I'm not misled by a fabricated answer	For an out-of-corpus question, the agent responds that it cannot answer from available sources, in at least 90% of a defined unanswerable test set.
US-4	Student	raise a new support ticket by describing my issue in my own words	I don't have to fill out a rigid form when self-service doesn't resolve it	Agent drafts a category + summary from free text and creates a ticket via the tool, returning a ticket ID.
US-5	Student	have the agent remember my active case for the rest of our conversation	I don't repeat my case ID every message	After one case-ID exchange, subsequent case-related questions in the same session resolve without re-asking for the ID.
US-6	Support-office staff	receive tickets already categorised and summarised	I can prioritise and act faster	Created tickets include a category, a short summary and the originating question/session reference.
US-7	Student	see which source a handbook answer came from	I can verify it myself if it matters	Every grounded answer displays or logs the source document/section used.
US-8	Quality/Security lead	have every agent action logged	we can audit decisions and debug failures later	Each tool call and model response is recorded with timestamp, inputs and outcome in a trace log.
US-9	Student	know upfront what the agent cannot help with	I don't waste time asking about admissions, grades or fees	A visible boundary statement lists excluded topics; agent redirects such questions to the correct human channel.
US-10	QA lead	have the agent refuse to invent a case status if the tool call fails	we never show a student false information	On tool failure/timeout, the agent reports the failure plainly instead of generating a plausible-looking status.
US-11	Student	get a plain-language answer rather than a dumped block of handbook text	the response is actually usable	Reviewer check: no single retrieved chunk is reproduced verbatim beyond a short supporting excerpt.
US-12	Project team	test the workflow against a small defined scenario set before scaling	we catch failures early and can defend the system at review	A scenario set covering normal, edge and unanswerable cases exists and is run before each weekly submission.

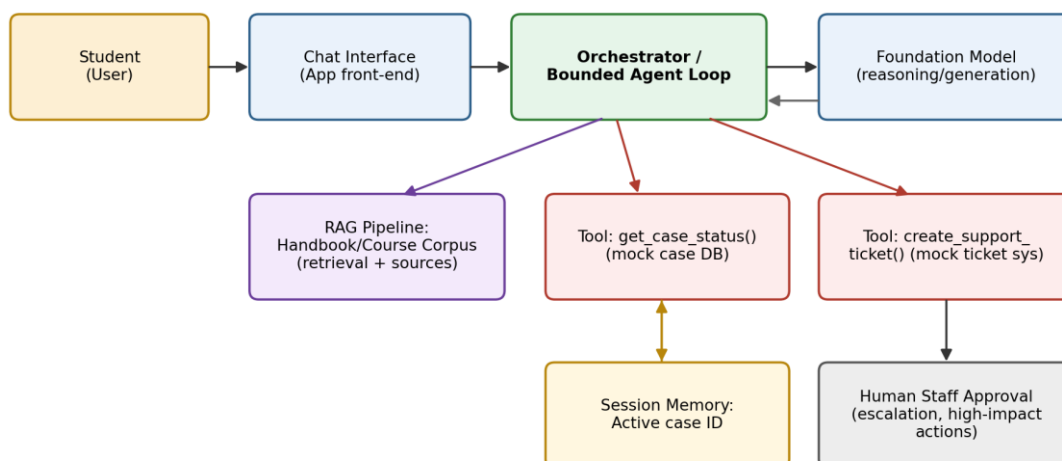
4. AI Boundary Matrix

Category	AI May Do	Stays Deterministic	Requires Approval	Human
Handbook / course Q&A	Generate a natural-language answer from retrieved evidence	Retrieval, indexing and source-validity checking are fixed pipeline code, not model output	—	
Case-status lookup	Explain/format the result in plain language	The actual status value always comes from the <code>get_case_status</code> tool/DB query, never generated	—	
Ticket drafting & creation	Draft category and summary from the student's free text	Ticket ID assignment, storage and routing rules are fixed software logic	Any ticket the routing rules flag as high-impact or an escalation is held for staff sign-off before final routing	
Session memory (active case)	Read/reference the stored active case ID within a session	Storage schema, retention period and deletion trigger are fixed by code, not the model	Clearing memory requires an explicit, deterministic user command	
Escalation / priority	May suggest a category	Priority thresholds and escalation rules are rule-based, not model-decided	Any case touching disciplinary, admissions or fee matters is routed to a human channel outside the agent entirely	
Admissions, grading, disciplinary, fee decisions	Not permitted at all	Entirely deterministic, human-run university processes	Always human — out of the agent's action space by design	

5. Initial Architecture / Context Diagram

This is the planned Week-1 shape of the system; tool implementations, the RAG pipeline and the agent loop itself are built progressively from Week 2 onward per the module timeline.

University Student-Support Case Agent — Week 1 Architecture / Context Diagram



Excluded entirely from agent scope: admissions, grading, disciplinary and fee decisions — human-only, deterministic channels.

6. Way Forward: Week 1 Roles and Task Breakdown

The brief specifies five roles for a five-person group; for a four-person team, merge Quality/Security with DevOps/Documentation as shown below. Every member still owns identifiable tasks and must be able to explain the group's decisions.

6.1 Role Assignment

Role	Owner	Week 1 Responsibility
Project/ Requirements Lead	Kabanda Arthur	Owens Project Charter, user stories, and coordination with the course convener; keeps ClickUp board current.
Application/Integration Lead	Tumukunde Andrew	Sets up the GitHub repository structure, README, .env.example, and initial app skeleton.
AI Engineering Lead	Ariko Sossy Joel	Drafts the AI Boundary Matrix content, begins scouting the model choice for Week 2, starts corpus scouting for Week 3.
DevOps/Documentation Lead	Garanga John	Sets up ClickUp workspace/tasks, reviews charter for testability, prepares the Week 1 progress report and repo documentation folders.

6.2 Immediate Task List (due 4 September 2026)

Task	Owner	Target
Confirm use case + fill in group name/members	Whole team	Today
Write/finalise Project Charter (this document, Section 2)	Requirements Lead	Today
Review and finalise 8–12 user stories + acceptance criteria	Requirements Lead + QA Lead	Today
Finalise AI Boundary Matrix wording for your specific group	AI Engineering Lead	Today
Create GitHub repo with the recommended folder structure (README, docs/, requirements/, architecture/, weekly-reports/, evaluation/, prompts/, knowledge/, src/, tests/, evidence/, .env.example)	Application/Integration Lead	Today
Create ClickUp project; add Week 1 tasks with owners and due dates; link tasks to repo	QA/DevOps Lead	Today
Commit Project Charter, user stories, AI Boundary Matrix, and architecture diagram to the repo (docs/ and architecture/)	Application/Integration Lead	Today
Start Week 1 progress report (1–2 pages): work completed, key decisions, challenges, links, individual contributions, next-week plan	Requirements Lead	Before submission
Do the Week 1 required reading (Osmani ch.1 pp.4-16, 30-38; Huyen ch.1 pp.16-47; Lanham ch.1 pp.1-11) — needed to defend decisions at review	Whole team	This week
Submit: Project Charter, user stories, AI Boundary Matrix, architecture diagram, GitHub+ClickUp evidence, Week 1 progress report	Whole team	Due 4 Sept 2026

GITHUB LINK: <https://github.com/Kabandaarthur/BSE4104.git>